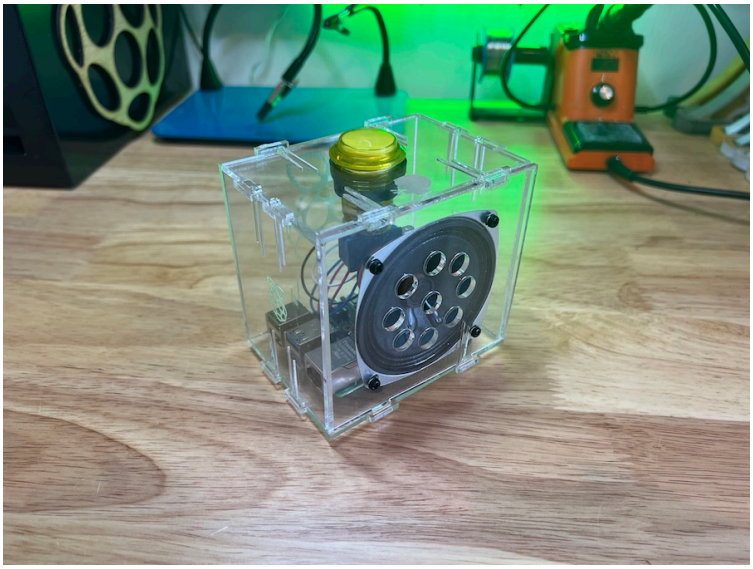
Raspberry Pi audio board getting started

Source: https://github.com/raspberrypi/documentation/blob/7376533ab86ea07acc77ef6a9f3c3e57fcff833d/documentation/asciidoc/accessories/audio/getting_started.adoc (CC BY-SA 4.0; adapted for this benchmark)
Image source: https://raw.githubusercontent.com/raspberrypi/documentation/7376533ab86ea07acc77ef6a9f3c3e57fcff833d/documentation/asciidoc/accessories/audio/images/Chatter_Box.jpg (CC BY-SA 4.0)



Raspberry Pi audio board getting started

Getting started

Create a toy chatter box

As an example of what Raspberry Pi Audio Boards can do, let's walk through the creation of a toy chatter box. Its on-board microphone, programmable button and speaker driver make the Codec Zero an ideal choice for this application.

A random pre-recorded five-second audio clip will be played when the button is pressed. After holding for ten seconds, a notifying burp sound will be emitted, after which a new five-second clip will be recorded. Holding the button down for more than 20 seconds will play a second burp sound, and then erase all previous recordings.

Hardware and wiring

For this project, any small passive speaker is probably sufficient. We're using one available here, which handles 5 W of power at 4 Ω. We have also used an illuminated momentary push button, and a laser-cut box to house all the components; but both are entirely optional. This example will work just using the Codec Zero's on-board button, which is pre-wired to GPIO 27. (Alternatively, you can use any momentary push button, such as those available here.)

Use a small flat-head screwdriver to attach your speaker to the screw terminals. For the additional push button, solder the button wires directly to the Codec Zero pads as indicated, using GPIO pin 27 and Ground for the switch, and +3.3V and Ground for the LED, if necessary.

Set up your Raspberry Pi

In this example, we are using Raspberry Pi OS Lite. Refer to our guide on installing Raspberry Pi OS for more details.

Make sure that you update your operating system before proceeding and follow the instructions provided for Codec Zero configuration, including the commands to enable the on-board microphone and speaker output.

Program your Raspberry Pi

Open a shell — for instance by connecting over SSH — on your Raspberry Pi and run the following to create our Python script:

$ sudo nano chatterbox.py

Add the following to the file, replacing <username> with your username:

```
#!/usr/bin/env python3
from gpiozero import Button
from signal import pause
import time
import random
import os
from datetime import datetime
# Print current date
date = datetime.now().strftime("%d%m%Y-%H:%M:%S")
print(f"{date}")
# Make sure that the 'sounds' folder exists, and if it does not, create it
path = '/home/<username>/sounds'
isExist = os.path.exists(path)
if not isExist:
os.makedirs(path)
print("The new directory is created!")
os.system('chmod 777 -R /home/<username>/sounds')
# Download a 'burp' sound if it does not already exist
burp = '/home/<username>/burp.wav'
isExist = os.path.exists(burp)
if not isExist:
os.system('wget http://rpf.io/burp -O burp.wav')
print("Burp sound downloaded!")
# Setup button functions - Pin 27 = Button hold time 10 seconds.
button = Button(27, holdtime=10)
def pressed():
global presstime
presstime = time.time()
print("Pressed at %s" % (presstime));
def released():
releasetime = time.time()
pressedfor = releasetime - presstime
print("Released at %s after %.2f seconds" % (releasetime, pressedfor))
if pressedfor < button.holdtime:
print("This is a short press")
randomfile = random.choice(os.listdir("/home/<username>/sounds/"))
file = '/home/<username>/sounds/' + randomfile
os.system('aplay ' + file)
```

```
elif pressedfor > 20:

os.system('aplay ' + burp)

print("Erasing all recorded sounds")

os.system('rm /home/<username>/sounds/');

def held():

print("This is a long press")

os.system('aplay ' + burp)

os.system('arecord --format S16LE --duration=5 --rate 48000 -c2 /home/<username>/sounds/$(date +"%d%m%Y-%H%M%S")voice.m4a');

button.whenpressed = pressed

button.whenreleased = released

button.whenheld = held

pause()
```

Press Ctrl+X, then the Y key, then Enter to save. To make the script executable, type the following:

```
$ sudo chmod +x chatterbox.py
```

Next, we need to create a crontab daemon that will automatically start the script each time the device is powered on. Run the following command to open your crontab for editing:

```
$ crontab -e
```

You're asked to select an editor; we recommend you use nano. Select it by entering the corresponding number, and press Enter to continue. Add the following line to the bottom of the file, replacing <username> with your username:

```
@reboot python /home/<username>/chatterbox.py
```

Press Ctrl X, then Y, then Enter to save, then reboot your device with sudo reboot.

Use the toy chatter box

The final step is to ensure that everything is operating as expected. Press the button and release it when you hear the burp. The recording will now begin for a period of five seconds. After you release the button, press it briefly again to hear the recording. Repeat this process as many times as you wish, and your sounds will be played at random. You can delete all recordings by pressing and holding the button, keeping the button pressed during the first burp and recording process, and releasing it after at least 20 seconds, at which point you will hear another burp sound confirming that the recordings have been deleted.

Next steps

Upgrades! It is always fun to upgrade a project, so why not add some additional features, such as an LED that will illuminate when recording? This project has all the parts required to make your own version of a Google intelligent speaker system, or you might want to consider building a second device that can be used to create a pair of walkie-talkies that are capable of transferring audio files over a network by using SSH.